MATERIA

INF761

Desarrollo de Videojuegos

Guía de Laboratorio 7

Platformer III: Peligro y Recompensa — Enemigos, Coleccionables y Daño

Carrera: Ingeniería Informática

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán

Semestre: I-2026

Universidad: Universidad Autónoma Tomás Frías (UATF)

Objetivos de Aprendizaje

1. Crear enemigos patrulleros con movimiento por waypoints y detección de bordes para inversión de dirección.
2. Implementar un sistema de coleccionables (monedas) con contador de puntaje y ScriptableObjects para definir datos de ítems.
3. Construir un sistema de vidas con daño por contacto, invulnerabilidad temporal (iframes) y feedback visual.
4. Agregar trampas estáticas (pinchos) como elementos de peligro ambiental en el nivel.

Prerrequisitos

- Guías 5–6 completadas: proyecto PlatformerGame con nivel de tilemap, personaje animado con movimiento pulido (Coyote Time, Jump Buffer, salto variable).
- Sprites de enemigos, monedas y trampas (incluidos en Pixel Adventure 1 o similares).
- Unity 6 LTS con el proyecto PlatformerGame abierto en la escena Level_01.

1. Marco Teórico

1.1 ScriptableObjects: Datos como Assets

Los ScriptableObjects son contenedores de datos que existen como assets en el proyecto, independientes de cualquier escena o GameObject. En lugar de definir valores (nombre, puntos, sprite) directamente en cada script de moneda o enemigo, se crean ScriptableObjects que centralizan esos datos. Si se necesita cambiar el valor de una moneda dorada de 5 a 10 puntos, se modifica un solo asset en lugar de buscar todos los GameObjects que lo usen.

```
C# — Patrón ScriptableObject
// Definición de un ScriptableObject
[CreateAssetMenu(
    fileName = "NewItemData",
    menuName = "Game/Item Data"
)]
public class ItemData : ScriptableObject
{
    public string itemName;
    public int pointValue;
    public Sprite icon;
    public AudioClip collectSound;
}

// Uso en un MonoBehaviour
public class Collectible : MonoBehaviour
{
    [SerializeField] private ItemData data;
    // data.pointValue, data.icon, etc.
}
```

Ventajas de ScriptableObjects

Separación de datos y lógica: los diseñadores pueden modificar valores sin tocar código.

Reutilización: el mismo ItemData puede usarse en monedas, gemas, estrellas, etc.

Sin duplicación: un asset compartido por múltiples instancias no se copia en memoria.

Versionable: al ser archivos, se rastrean fácilmente con Git.

1.2 Enemigos Patrulleros con Waypoints

Un patrón común para enemigos en platformers es el patrullaje entre puntos predefinidos (waypoints). El enemigo se mueve hacia el siguiente waypoint; al llegar, cambia de dirección hacia el anterior. Para enemigos que patrullan sobre plataformas, se puede combinar con detección de borde usando un Raycast hacia abajo desde el frente del enemigo: si no detecta suelo, invierte la dirección para no caer.

1.3 Sistema de Daño e Invulnerabilidad (iframes)

Cuando el jugador recibe daño, se activa un período de invulnerabilidad temporal (invincibility frames o iframes) durante el cual no puede recibir más daño. Esto evita que el jugador pierda múltiples vidas en un instante por estar en contacto continuo con un enemigo. Durante los iframes, el sprite parpadea para dar feedback visual al jugador.

Concepto	Implementación
Daño	OnTriggerEnter2D o OnCollisionEnter2D detecta contacto con enemigo/trampa. Reduce vidas en 1.
iframes	Coroutine que activa un flag isInvulnerable por N segundos. Mientras está activo, ignora daño.
Parpadeo	Dentro de la Coroutine, alternar el SpriteRenderer.enabled (on/off) cada 0.1s para crear efecto visual.
Knockback	Al recibir daño, aplicar una fuerza opuesta a la dirección del enemigo para empujar al jugador.
Muerte	Si vidas llegan a 0, desactivar controles y reiniciar posición (o recargar escena en la Guía 8).

1.4 Trampas Estáticas

Las trampas como pinchos son elementos ambientales que dañan al jugador por contacto. A diferencia de los enemigos, no tienen movimiento ni IA: son colliders con trigger que aplican daño. Se pueden construir con tiles especiales en una capa dedicada del Tilemap, o como GameObjects individuales con un BoxCollider2D marcado como trigger.

2. Desarrollo Paso a Paso

Continuar con el proyecto **PlatformerGame** de las Guías 5–6. Abrir la escena Level_01.

1 Crear el ScriptableObject para Datos de Ítems

Definir la estructura de datos reutilizable para coleccionables:

1.1 Script ItemData

Crear el script Assets/Scripts/ItemData.cs:

```
C# — ItemData.cs
using UnityEngine;

[CreateAssetMenu(
    fileName = "NewItemData",
    menuName = "Game/Item Data"
)]
public class ItemData : ScriptableObject
{
    [Header("Información General")]
    public string itemName;
    [TextArea] public string description;

    [Header("Gameplay")]
    public int pointValue = 1;

    [Header("Visual")]
    public Sprite icon;
    public Color particleColor = Color.yellow;

    [Header("Audio")]
    public AudioClip collectSound;
}
```

1.2 Crear assets de datos

5. En el Project, clic derecho > Create > Game > Item Data.
6. Crear tres assets de ejemplo:
   - CoinBronze — pointValue: 1, color bronce
   - CoinSilver — pointValue: 5, color plata
   - CoinGold — pointValue: 10, color dorado
7. Asignar el sprite y sonido correspondiente a cada uno.
8. Guardar los assets en Assets/Data/.

CreateAssetMenu

El atributo [CreateAssetMenu] agrega la opción al menú contextual del Project, permitiendo crear instancias del ScriptableObject como si fueran cualquier otro asset. Sin este atributo, habría que crearlos manualmente desde Assets > Create > ScriptableObject.

2 Implementar el Sistema de Coleccionables

Crear monedas que el jugador pueda recoger:

2.1 Script del coleccionable

Crear Assets/Scripts/Collectible.cs:

```
C# — Collectible.cs
using UnityEngine;

public class Collectible : MonoBehaviour
{
    [SerializeField] private ItemData data;

    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            // Sumar puntos
            GameManager.Instance.AddScore(data.pointValue);

            // Reproducir sonido
            if (data.collectSound != null)
            {
                AudioSource.PlayClipAtPoint(
                    data.collectSound,
                    transform.position
                );
            }

            // Destruir el coleccionable
            Destroy(gameObject);
        }
    }
}
```

2.2 GameManager (Singleton básico)

Crear Assets/Scripts/GameManager.cs para gestionar el estado del juego:

```
C# — GameManager.cs
using UnityEngine;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }

    [Header("Game State")]
    [SerializeField] private int maxLives = 3;

    private int currentScore;
    private int currentLives;

    public int Score => currentScore;
    public int Lives => currentLives;

    private void Awake()
    {
        // Singleton: solo una instancia
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;

        currentLives = maxLives;
        currentScore = 0;
    }

    public void AddScore(int points)
    {
        currentScore += points;
        Debug.Log($"Score: {currentScore}");
        // En la Guía 8 se conectará con la UI
    }

    public void TakeDamage()
    {
        currentLives--;
        Debug.Log($"Lives: {currentLives}");

        if (currentLives <= 0)
        {
            GameOver();
```

```
        }
    }

    public void ResetLives()
    {
        currentLives = maxLives;
    }

    private void GameOver()
    {
        Debug.Log("GAME OVER");
        // En la Guía 8 se mostrará pantalla de Game Over
        // Por ahora, reiniciar posición y vidas
        currentLives = maxLives;
        currentScore = 0;
    }
}
```

2.3 Configurar el coleccionable en la escena

9. Crear un GameObject vacío llamado Coin.
10. Agregar un SpriteRenderer con el sprite de la moneda.
11. Agregar un CircleCollider2D, marcar Is Trigger.
12. Agregar el script Collectible y asignar el ItemData correspondiente (ej: CoinBronze).
13. Crear un Prefab: arrastrar el Coin a Assets/Prefabs/.
14. Colocar varias instancias del prefab a lo largo del nivel.

2.4 Configurar el GameManager

15. Crear un GameObject vacío llamado GameManager en la Hierarchy.
16. Agregar el script GameManager.cs.
17. Verificar que el tag Player esté asignado al GameObject Player.

Animación de monedas

Para que las monedas se vean más atractivas, crear un Animation Clip simple con los frames de giro de la moneda (si el tileset lo incluye). Agregar un componente Animator al prefab con este clip en loop.

3 Implementar el Sistema de Vidas, Daño e Iframes

Agregar la capacidad de recibir daño con invulnerabilidad temporal:

3.1 Script PlayerHealth

Crear Assets/Scripts/PlayerHealth.cs:

```
C# — PlayerHealth.cs
using System.Collections;
using UnityEngine;

public class PlayerHealth : MonoBehaviour
{
    [Header("Invulnerability")]
    [SerializeField] private float iframeDuration = 1.5f;
    [SerializeField] private float flashInterval = 0.1f;

    [Header("Knockback")]
    [SerializeField] private float knockbackForce = 8f;
    [SerializeField] private float knockbackDuration = 0.2f;

    private SpriteRenderer spriteRenderer;
    private Rigidbody2D rb;
    private PlayerController playerController;
    private bool isInvulnerable;

    private void Awake()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        rb = GetComponent<Rigidbody2D>();
        playerController =
            GetComponent<PlayerController>();
    }

    public void TakeDamage(Vector2 damageSource)
    {
        if (isInvulnerable) return;

        // Registrar daño en el GameManager
        GameManager.Instance.TakeDamage();

        // Iniciar iframes con parpadeo
        StartCoroutine(InvulnerabilityCoroutine());

        // Aplicar knockback
        StartCoroutine(
            KnockbackCoroutine(damageSource)
        );
    }

    private IEnumerator InvulnerabilityCoroutine()
    {
        isInvulnerable = true;
```

```
        // Parpadeo visual
        float elapsed = 0f;
        while (elapsed < iframeDuration)
        {
            spriteRenderer.enabled =
                !spriteRenderer.enabled;
            yield return new WaitForSeconds(
                flashInterval
            );
            elapsed += flashInterval;
        }

        // Asegurar que el sprite quede visible
        spriteRenderer.enabled = true;
        isInvulnerable = false;
    }

    private IEnumerator KnockbackCoroutine(
        Vector2 damageSource)
    {
        // Desactivar control del jugador brevemente
        playerController.enabled = false;

        // Dirección opuesta al daño
        Vector2 knockDir = (
            (Vector2)transform.position - damageSource
        ).normalized;

        rb.linearVelocity = new Vector2(
            knockDir.x * knockbackForce,
            knockbackForce * 0.5f  // Empuje hacia arriba
        );

        yield return new WaitForSeconds(
            knockbackDuration
        );

        // Reactivar control
        playerController.enabled = true;
    }
}
```

3.2 Agregar el componente al Player

18. Seleccionar el Player en la Hierarchy.
19. Agregar el script PlayerHealth.
20. Configurar los valores en el Inspector:

Campo	Valor Sugerido
`Iframe Duration`	1.5
`Flash Interval`	0.1
`Knockback Force`	8
`Knockback Duration`	0.2

playerController.enabled

Desactivar el PlayerController durante el knockback impide que el jugador cancele la fuerza de retroceso con su input. Esto se siente más natural y enfatiza el impacto del daño. La duración debe ser breve (0.15–0.25s) para no frustrar al jugador.

4 Crear Enemigos Patrulleros

Implementar enemigos que patrullan entre dos puntos y dañan al jugador:

4.1 Script EnemyPatrol

Crear Assets/Scripts/EnemyPatrol.cs:

```
C# — EnemyPatrol.cs
using UnityEngine;

public class EnemyPatrol : MonoBehaviour
{
    [Header("Patrol")]
    [SerializeField] private Transform pointA;
    [SerializeField] private Transform pointB;
    [SerializeField] private float moveSpeed = 2f;

    [Header("Edge Detection")]
    [SerializeField] private Transform edgeCheck;
    [SerializeField] private float edgeCheckDistance = 1f;
    [SerializeField] private LayerMask groundLayer;

    private Rigidbody2D rb;
    private SpriteRenderer spriteRenderer;
    private Animator animator;
    private Vector2 currentTarget;
    private bool facingRight = true;
```

```
    private void Awake()
    {
        rb = GetComponent<Rigidbody2D>();
        spriteRenderer = GetComponent<SpriteRenderer>();
        animator = GetComponent<Animator>();
    }

    private void Start()
    {
        currentTarget = pointB.position;
    }

    private void FixedUpdate()
    {
        Patrol();
        CheckEdge();
    }

    private void Patrol()
    {
        // Mover hacia el objetivo actual
        Vector2 direction = (
            currentTarget - (Vector2)transform.position
        ).normalized;

        rb.linearVelocity = new Vector2(
            direction.x * moveSpeed,
            rb.linearVelocity.y
        );

        // Verificar si llegó al waypoint
        float distance = Vector2.Distance(
            transform.position, currentTarget
        );
        if (distance < 0.2f)
        {
            // Cambiar de objetivo
            currentTarget =
                currentTarget == (Vector2)pointA.position
                    ? pointB.position
                    : pointA.position;

            Flip();
        }
    }

    private void CheckEdge()
    {
        // Raycast hacia abajo desde el frente
        if (edgeCheck == null) return;
```

```
        RaycastHit2D hit = Physics2D.Raycast(
            edgeCheck.position,
            Vector2.down,
            edgeCheckDistance,
            groundLayer
        );

        // Si no detecta suelo, invertir
        if (!hit.collider)
        {
            currentTarget =
                currentTarget == (Vector2)pointA.position
                    ? pointB.position
                    : pointA.position;

            Flip();
        }
    }

    private void Flip()
    {
        facingRight = !facingRight;
        transform.localScale = new Vector3(
            facingRight ? 1f : -1f, 1f, 1f
        );
    }

    // Visualizar waypoints y edge check en el editor
    private void OnDrawGizmosSelected()
    {
        if (pointA != null && pointB != null)
        {
            Gizmos.color = Color.cyan;
            Gizmos.DrawSphere(pointA.position, 0.2f);
            Gizmos.DrawSphere(pointB.position, 0.2f);
            Gizmos.DrawLine(
                pointA.position, pointB.position
            );
        }
        if (edgeCheck != null)
        {
            Gizmos.color = Color.red;
            Gizmos.DrawRay(
                edgeCheck.position,
                Vector2.down * edgeCheckDistance
            );
        }
    }
}
```

4.2 Script EnemyDamage

Crear Assets/Scripts/EnemyDamage.cs para manejar el daño al contacto:

```
C# — EnemyDamage.cs
using UnityEngine;

public class EnemyDamage : MonoBehaviour
{
    private void OnCollisionEnter2D(
        Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Player"))
        {
            PlayerHealth health =
                collision.gameObject
                    .GetComponent<PlayerHealth>();

            if (health != null)
            {
                health.TakeDamage(
                    transform.position
                );
            }
        }
    }
}
```

4.3 Configurar el enemigo en la escena

21. Crear un GameObject vacío llamado Enemy_Slime (o el nombre del enemigo elegido).
22. Agregar SpriteRenderer con el sprite del enemigo.
23. Agregar Animator con un clip de movimiento en loop (si hay sprite sheet del enemigo).
24. Agregar Rigidbody2D: Dynamic, Freeze Rotation Z activado, Gravity Scale: 3.
25. Agregar BoxCollider2D o CapsuleCollider2D ajustado al sprite.
26. Agregar los scripts EnemyPatrol y EnemyDamage.

27. Configurar los waypoints:
    - Crear dos GameObjects hijos vacíos: WaypointA y WaypointB.
    - Posicionar WaypointA en el extremo izquierdo de la patrulla.
    - Posicionar WaypointB en el extremo derecho.
    - En el Inspector del EnemyPatrol, asignar Point A y Point B.

28. Configurar el edge check:
    - Crear un GameObject hijo vacío: EdgeCheck.
    - Posicionarlo frente al enemigo, ligeramente debajo (al nivel de los pies).

- Asignar la referencia en el script y el Ground Layer.

29. Crear un Prefab del enemigo para reutilizar.

5 Implementar Trampas Estáticas (Pinchos)

Agregar peligros ambientales al nivel:

5.1 Script DamageZone

Crear un script genérico Assets/Scripts/DamageZone.cs que se puede reutilizar para pinchos, lava, etc.:

```
C# — DamageZone.cs
using UnityEngine;

public class DamageZone : MonoBehaviour
{
    private void OnTriggerEnter2D(
        Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            PlayerHealth health =
                collision.GetComponent<PlayerHealth>();

            if (health != null)
            {
                health.TakeDamage(
                    transform.position
                );
            }
        }
    }
}
```

5.2 Colocar pinchos en el nivel

Opción A — Como GameObjects individuales:

30. Crear un GameObject con SpriteRenderer (sprite de pinchos).
31. Agregar BoxCollider2D marcado como Is Trigger, ajustado a la zona de daño.
32. Agregar el script DamageZone.
33. Crear prefab y distribuir en el nivel.

Opción B — Como capa de Tilemap (más eficiente para muchos pinchos):

34. Crear un nuevo Tilemap hijo del Grid llamado Hazards.
35. Pintar los tiles de pinchos en esta capa.
36. Agregar Tilemap Collider 2D al Tilemap Hazards.
37. Marcar Is Trigger en el Tilemap Collider 2D.
38. Agregar el script DamageZone al GameObject Hazards.

Tilemap como Trigger

Cuando el Tilemap Collider 2D está marcado como Is Trigger, no genera colisión física (el jugador puede pasar a través). Solo detecta el contacto para aplicar daño. Esto es ideal para pinchos donde el jugador debe poder tocarlos y ser empujado por el knockback.

6 Actualizar la KillZone con el Sistema de Vidas

Refactorizar la KillZone de la Guía 5 para que use el nuevo sistema de vidas:

```
C# — KillZone.cs (actualizado)
using UnityEngine;

public class KillZone : MonoBehaviour
{
    [SerializeField] private Transform respawnPoint;

    private void OnTriggerEnter2D(
        Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            // Quitar una vida
            GameManager.Instance.TakeDamage();

            // Reposicionar al jugador
            collision.transform.position =
                respawnPoint.position;

            // Resetear velocidad
            Rigidbody2D rb =
                collision.GetComponent<Rigidbody2D>();
            if (rb != null)
            {
                rb.linearVelocity = Vector2.zero;
            }
        }
    }
}
```

39. Crear un GameObject vacío llamado RespawnPoint y posicionarlo al inicio del nivel.
40. En el Inspector de la KillZone, asignar el RespawnPoint al campo correspondiente.
41. La KillZone ahora usa GameManager.TakeDamage() en lugar de reposicionar directamente.

RespawnPoint

Usar una referencia a un Transform para el punto de reaparición es más flexible que hardcodear coordenadas. En la Guía 8 se implementará un sistema de checkpoints donde el respawn cambia dinámicamente.

3. Ejercicios Propuestos

Ejercicio 1: Enemigo Eliminable

Implementar la mecánica clásica de eliminar enemigos al saltar sobre ellos. Detectar si el jugador colisiona con el enemigo desde arriba (comparando las posiciones Y). Si el impacto es desde arriba, destruir al enemigo y dar un pequeño impulso de salto al jugador (bounce). Si es desde el costado o abajo, aplicar daño al jugador.

Ejercicio 2: Plataformas Móviles

Crear una plataforma que se mueva entre dos puntos (horizontal o vertical) usando el mismo patrón de waypoints del enemigo. El jugador debe poder pararse y moverse con la plataforma. Pista: hacer al jugador hijo temporal de la plataforma al aterrizar (transform.SetParent) y desvincularlo al saltar.

Ejercicio 3: Segundo Tipo de Coleccionable

Utilizando el sistema de ScriptableObjects, crear un nuevo tipo de coleccionable (ej: gema, estrella, corazón de vida extra). Crear un nuevo ItemData con valores distintos. Para el corazón, en lugar de sumar puntos, agregar un método AddLife() al GameManager que incremente currentLives (sin exceder maxLives).

4. Bibliografía y Recursos

- *Unity Technologies.* Unity Manual — ScriptableObject. https://docs.unity3d.com/Manual/class-ScriptableObject.html
- *Unity Technologies.* Unity Manual — Coroutines. https://docs.unity3d.com/Manual/Coroutines.html
- *Unity Technologies.* Unity Manual — Physics2D.Raycast. https://docs.unity3d.com/ScriptReference/Physics2D.Raycast.html

- *Unity Technologies.* Unity Manual — Collider2D. https://docs.unity3d.com/Manual/Collider2D.html
- *Pixel Frog.* Pixel Adventure 1 — Free Asset Pack. https://pixelfrog-assets.itch.io/pixel-adventure-1
- *Hocking, J.* Unity in Action, Third Edition. Manning Publications, 2022.